

@ Grupo 20

INFORME TECNICO
ANALITICO

TP Integrador

Integrante:

Roca Santiago Nicolas

2026

ERROR 1: Funcion Auditoria recursión Infinita

Causa: La funcion auditoria-IT2 recibia un argumento color-anterior que no pertenecia a los colores de un semaforo (por ejemplo, el simbolo 'azul'). Al evaluar el predicado de transicion vial, la funcion siempre ingresaba a la rama por defecto (t) y volviendo a llamarse recursivamente con los mismos datos invalidos de manera infinita.

```
"Error datos no numericos"
0] (auditoria-IT2 'rojo 1 1 1 3)

(("[2026-06-14 17:07:01]" ROJO ROJO-INTERMITENTE)
 ("[2026-06-14 17:07:04]" ROJO-INTERMITENTE VERDE)
 ("[2026-06-14 17:07:05]" VERDE VERDE-INTERMITENTE))
0] (auditoria-IT2 'rojo 1 1 'a 3)

"Error: No se puede iniciar la auditoria con datos invalidos"
0] (auditoria-IT2 'rojo 1 1 1 'a)

"Error: No se puede iniciar la auditoria con datos invalidos"
0] (auditoria-IT2 'azul 1 1 1 3)
```

Solucion: diseñe un mecanismo de defensa explicito en la primera rama del cond. Defini una lista de colores-validos en el cual se encontraban todos los colores que si eran validos para que luego con las funciones (not (member color-anterior colores-validos)) verificar que este dentro de esa lista ya que si el color ingresado no estaba en esa lista mi member iba a retornar nil y con mi not se iba a volver true, dando así mi mensaje de error.

```
(defun auditoria-IT2 (color-anterior duracion_rojo duracion_verde duracion_amarillo cambios)
  (let* ((color-nuevo (timer-IT2 (obtener-timestamp) duracion_rojo duracion_verde duracion_amarillo))
        (colores-validos '(rojo rojo-intermitente verde verde-intermitente amarillo amarillo-intermitente)))
    (cond ((or
            (stringp color-anterior)
            (not (duracion-ciclo-IT2 duracion_rojo duracion_verde duracion_amarillo))
            (not (numberp cambios)))
          "Error: No se puede iniciar la auditoria con datos invalidos")
          ((= cambios 0) nil) ;finalizacion de mi funcion recursiva
          ((validar-transiciones-p-IT2 (convertir-color-IT2 color-anterior) color-nuevo) ;Caso de encontrar una transicion
           (sleep 1)
           (cons (list (format nil "[%a]" (obtener-timestamp-humano)) color-anterior color-nuevo)
                 (auditoria-IT2 color-nuevo duracion_rojo duracion_verde duracion_amarillo (- cambios 1))))
          (t (sleep 1) ; caso para volver a llamarse
              (auditoria-IT2 color-anterior duracion_rojo duracion_verde duracion_amarillo cambios)))
    )
  )
)
```

ERROR 2: Error de validación de transiciones

Problema

Cuando quise probar mi función auditoria me confundí y le pase por ejemplo 'rojo en vez de 'en-rojo lo que hizo que mi función auxiliar llamada validar-transiciones-p-IT2 no diera como valida la transición y esto hiciera que no mostrara las transiciones.

Solucion

Entonces cree una función auxiliar la cual se llama convertir-color-IT2 que lo que hace es convertir mi 'rojo a 'en-rojo y así mi validar-transiciones-p-IT2 validara correctamente y muestre dichas transiciones, haciendo también que tranquilamente pueda llamar a mi función timer en la parte de color-anterior y que corra sin ningún problema.

```
(defun convertir-color-IT2 (color)
  (cond
    ((equal color 'rojo) 'en-rojo)
    ((equal color 'rojo-intermitente) 'en-rojo-intermitente)

    ((equal color 'verde) 'en-verde)
    ((equal color 'verde-intermitente) 'en-verde-intermitente)

    ((equal color 'amarillo) 'en-amarillo)
    ((equal color 'amarillo-intermitente) 'en-amarillo-intermitente)

    (t color)
  )
)
```

ERROR 3: Conflicto de Dependencia Lineal en la Compilacion Monolitica de SBCL

CAUSA

Al migrar a SBCL, el compilador me tiraba error porque funciones como timer-IT2 o ciclos-por-tiempo llamaban internamente a duracion-ciclo-IT2, la cual estaba declarada mas abajo en el archivo. GNU CLISP toleraba este desorden, pero SBCL realiza una lectura secuencial estricta y frena la ejecucion si se intenta invocar una funcion que todavia no fue leida ni cargada en memoria.

```
CONVERTIR-COLOR
* VALIDAR-TRANSICIONES-P
* TRANSICION
* OBTENER-TIMESTAMP
; in: DEFUN FTIMER
      (DURACION-CICLO DURACION_ROJO DURACION_VERDE DURACION_AMARILLO)
caught STYLE-WARNING:
  undefined function: COMMON-LISP-USER::DURACION-CICLO
compilation unit finished
  Undefined function:
    DURACION-CICLO
  caught 1 STYLE-WARNING condition
* INFORME-DISTRIBUCION
```

SOLUCION

Se reestructuro por completo el orden fisico del codigo fuente. Movi todas las funciones de soporte matematico y traduccion (como duracion-ciclo-IT2 y convertir-color-IT2) al principio del codigo, en la seccion de "FUNCIONES AUXILIARES".

```
DURACION-CICLO
* CONVERTIR-COLOR
* VALIDAR-TRANSICIONES-P
* TRANSICION
* OBTENER-TIMESTAMP
* FTIMER
* AUDITORIA
* RECOMENDACION-CICLO
* CICLOS-POR-TIEMPO
* INFORME-DISTRIBUCION
```

Al emproljar el orden, me asegure de que el compilador las conozca de antemano antes de ser llamadas por las funciones principales, solucionando el problema. las funciones puras de soporte matematico y calculo determinista (como duracion-ciclo-IT2 y convertir-color-IT2) al principio del archivo bajo la seccion de "FUNCIONES AUXILIARES", garantizando que esten cargadas en memoria antes de ser invocadas por las funciones principales.

JUSTIFICACION DEL USO DE LOCAL-TIME

La inclusion de la biblioteca local-time se debio puramente a una necesidad de **legibilidad humana**.

Al usar unicamente las funciones nativas de Common Lisp, el sistema registraba el tiempo en formato Unix Timestamp (un numero entero gigante que representa los segundos transcurridos desde 1970). Si bien este numero es excelente para que la computadora realice calculos matematicos, para una persona es imposible saber a que día o a que hora corresponde cuando mira la lista de registros que arroja la funcion auditoria-IT2.

Con la incorporacion de local-time, pude formatear ese numero de manera automatica. Gracias a esto, cada vez que el semaforo cambia de luz, la bitacora guarda una cadena de texto simple y clara (por ejemplo, "2026-06-15 18:01:10").

ERROR 4: INCOMPATIBILIDAD DE QUICKLISP

Problema

Durante la instalación del gestor de paquetes Quicklisp en GNU CLISP, se produjeron errores repetidos de conexión. El analisis determino que esta implementacion antigua posee limitaciones en su sistema de red y sockets nativos, lo que le impidio establecer conexiones seguras (SSL/TLS) para descargar bibliotecas modernas como local-time.

Solucion

Para resolverlo, se migro todo el proyecto a SBCL (Steel Bank Common Lisp), una implementacion moderna y de alto rendimiento. En SBCL, Quicklisp se cargo correctamente de forma inmediata, permitiendo la descarga normal de las dependencias necesarias para el semaforo.

INTEGRACION EXITOSA DE LOCAL-TIME EN SBCL

Como se aprecia en la captura, SBCL cargo e inicializo correctamente la biblioteca local-time mediante Quicklisp sin arrojar errores. Esto confirma que la migracion de compilador soluciono de raiz las limitaciones de red de GNU CLISP, garantizando un entorno estable y moderno para la auditoria y persistencia de datos del sistema.

```
This is SBCL 2.6.5, an implementation of ANSI Common Lisp.
More information about SBCL is available at <http://www.sbcl.org/>.

SBCL is free software, provided as is, with absolutely no warranty.
It is mostly in the public domain; some portions are provided under
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.
* (Load #P"C:/Users/WinterOS/quicklisp/setup.lisp")

(ql:quickload "local-time")

t
* To load "local-time":
  Load 1 ASDF system:
    local-time
; Loading "local-time"

("local-time")
* |
```

Estudio comparativo - Clojure

Clojure es un lenguaje de programación de propósito general que pertenece a la familia de los dialectos de Lisp. Se ejecuta sobre la Máquina Virtual de Java (JVM), lo que le permite una interoperabilidad total con todo el ecosistema de Java. Su diseño está fuertemente orientado al paradigma funcional y se destaca por tratar la gestión de estados mediante estructuras de datos inmutables por defecto, lo que facilita de forma nativa la programación concurrente y asíncrona sin los problemas clásicos de bloqueo de hilos de ejecución.

Áreas de Aplicación e Impacto en la Industria

- **Sistemas Financieros y Fintech:** Al garantizar la inmutabilidad de los datos, es ideal para procesar transacciones masivas donde los registros históricos no deben ser alterados.
- **Procesamiento de Datos en Tiempo Real y Big Data:** Su alta eficiencia en la JVM y su manejo asíncrono nativo lo posicionan fuertemente en la ingeniería de datos.
- **Desarrollo Web de Alto Rendimiento:** Tanto en el backend como en el frontend (a través de ClojureScript).

Empresas Globales que Utilizan Clojure

Nubank

El banco digital más grande de Latinoamérica basa casi toda su arquitectura de microservicios y procesamiento financiero en Clojure.

Walmart

Utiliza este lenguaje para gestionar sus sistemas de bases de datos distribuidas y procesamiento de eventos en temporadas de alta demanda.

Cisco

Implementa soluciones de gestión de redes y configuraciones seguras aprovechando la estabilidad del lenguaje sobre la JVM.

Preguntas - Clojure

1. ¿Cuál es la ventaja técnica y de rendimiento de un Keyword en Clojure para mapear estados?

En la version original en Common Lisp se utilizaban simbolos cotizados ('rojo, 'verde) para representar los estados de la maquina. Al migrar el nucleo logico a Clojure, se reemplazaron por Keywords (:rojo, :amarillo y :verde), cumpliendo con las buenas practicas de diseño de este lenguaje.

La ventaja tecnica principal es que las Keywords se internan en memoria una sola vez, lo que permite que Clojure compare los estados del semaforo mediante comparacion directa de punteros a nivel de la JVM (lo que optimiza drasticamente el rendimiento). Ademas de facilitar su integracion como claves en mapas de datos persistentes, eliminan el uso de la comilla de cotizacion, haciendo que el código de la auditoria quede mas legible, inmune a errores de evaluacion y facil de mantener.

2. ¿Cómo maneja Clojure el paso del tiempo en el simulador semaforico sin modificar variables?

Clojure prohíbe la mutabilidad tradicional (como el uso de `setq` o `setf` en Common Lisp) mediante el uso de Estructuras de Datos Persistentes. Cuando una estructura "cambia", Clojure no sobrescribe la memoria existente, sino que crea una nueva version de la estructura compartiendo de forma eficiente los nodos que no cambiaron

Para manejar el paso del tiempo en el simulador sin modificar variables, la funcion `timer-IT2` recibe el tiempo como un argumento inmutable (`ts`) y calcula la fase de las luces de forma puramente matematica con el operador `mod`. El tiempo avanza porque el entorno le pasa valores nuevos a la funcion en cada ciclo, pero la memoria de la computadora nunca se sobrescribe.

link de la pagina que utilice para ayudarme con clojure:

<https://cljs.info/cheatsheet/>